Time series Training and presentation

loading required libraries

In [150…
```
import pandas as pd
import seaborn as sns
import matplotlib .pyplot as plt
import numpy as np
import matplotlib.dates as mdates
import warnings
warnings.filterwarnings("ignore")
plt.style.use("fivethirtyeight")
import matplotlib.style as style
style.available
df = pd.read_csv("commodity_futures.csv",
                 parse_dates=["Date"], index_col=["Date"])
df.head()
```

Out[150]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	ZINC	ULS DIESEL	NICKEL	WHEAT	SUGAR	GASOLINE	COFFEE	LEAN HOGS	HRW WHEAT	COTTON
2000-01-03	NaN	NaN	NaN	NaN	456.50	200.75	NaN	NaN	NaN	69.700	...	1237.50	NaN	8446.0	247.50	6.10	NaN	116.50	55.975	274.25	51.07
2000-01-04	2.176	283.7	25.55	24.39	464.25	203.00	0.8480	5.375	213.50	69.075	...	1215.00	67.78	8314.0	247.25	5.77	NaN	116.25	55.625	274.00	50.73
2000-01-05	2.168	282.1	24.91	23.73	469.25	203.00	0.8565	5.210	213.00	68.975	...	1208.25	66.55	8307.0	249.75	5.81	NaN	118.60	55.975	276.25	51.56
2000-01-06	2.196	282.4	24.78	23.62	468.00	203.75	0.8530	5.167	211.25	70.075	...	1212.00	66.28	8252.0	248.50	5.77	NaN	116.85	55.175	276.00	52.08
2000-01-07	2.173	282.9	24.22	23.09	471.50	207.00	0.8540	5.195	205.25	70.875	...	1212.00	64.75	8474.0	251.75	5.84	NaN	114.15	55.625	277.75	52.96

5 rows × 23 columns

In [151…
```
df.shape
```
Out[151]:
```
(6092, 23)
```

data wrangling techniques

In [152…
```
df.index
```
Out[152]:
```
DatetimeIndex(['2000-01-03', '2000-01-04', '2000-01-05', '2000-01-06',
               '2000-01-07', '2000-01-10', '2000-01-11', '2000-01-12',
               '2000-01-13', '2000-01-14',
               ...
               '2023-07-24', '2023-07-25', '2023-07-26', '2023-07-27',
               '2023-07-28', '2023-07-31', '2023-08-01', '2023-08-02',
               '2023-08-03', '2023-08-04'],
              dtype='datetime64[ns]', name='Date', length=6092, freq=None)
```

In [153…
```
df.head(1)
```
Out[153]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	ZINC	ULS DIESEL	NICKEL	WHEAT	SUGAR	GASOLINE	COFFEE	LEAN HOGS	HRW WHEAT	COTTON
2000-01-03	NaN	NaN	NaN	NaN	456.5	200.75	NaN	NaN	NaN	69.7	...	1237.5	NaN	8446.0	247.5	6.1	NaN	116.5	55.975	274.25	51.07

1 rows × 23 columns

In [154…
```
df.shape
```
Out[154]:
```
(6092, 23)
```

In [155…
```
df.drop(["GASOLINE"],axis= 1 ,inplace = True)
```

In [156…
```
df.head(1)
```
Out[156]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	SOYBEAN MEAL	ZINC	ULS DIESEL	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON
2000-01-03	NaN	NaN	NaN	NaN	456.5	200.75	NaN	NaN	NaN	69.7	...	146.4	1237.5	NaN	8446.0	247.5	6.1	116.5	55.975	274.25	51.07

1 rows × 22 columns

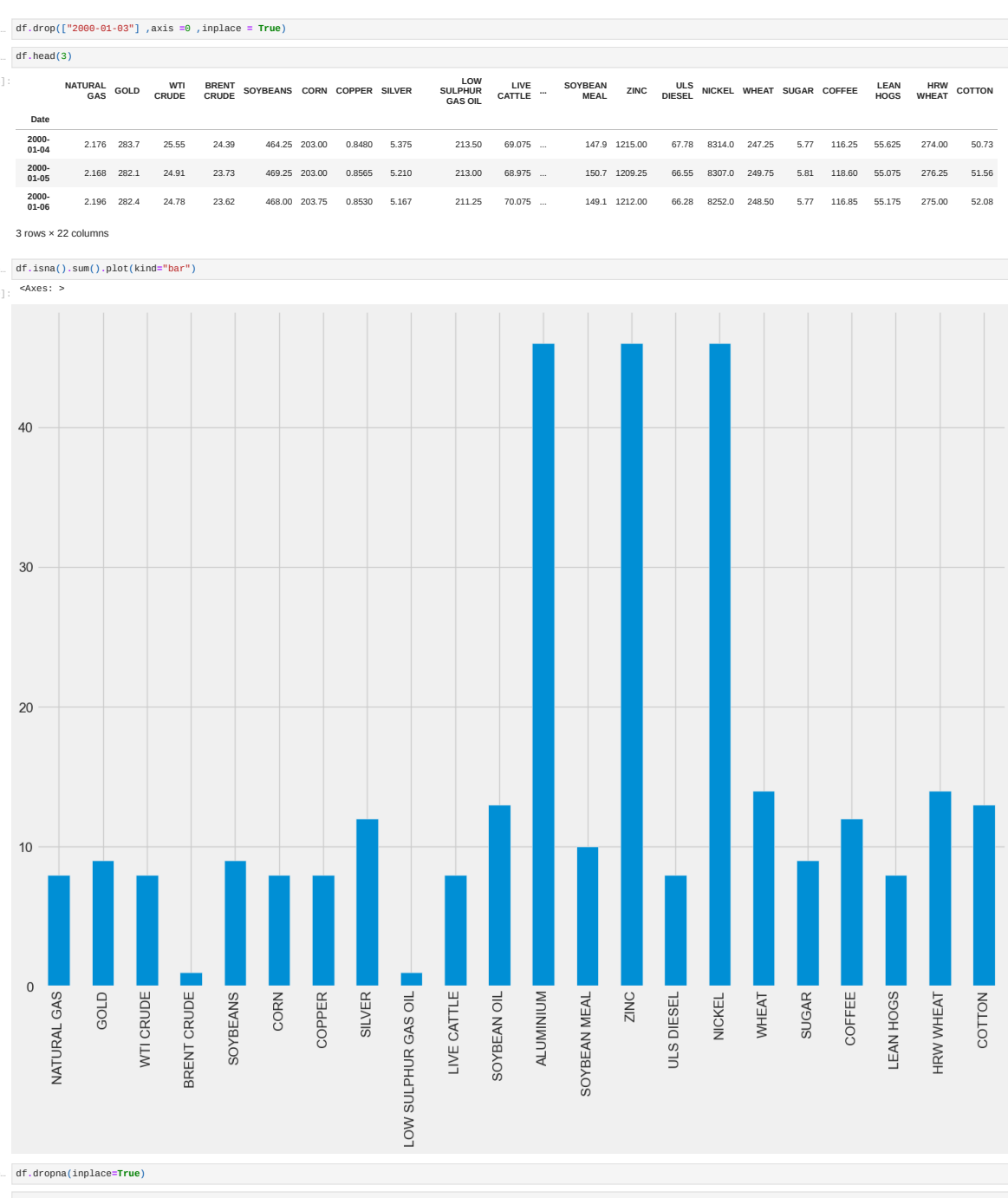
In [157…
```
df.drop(["2000-01-03"] ,axis =0 ,inplace = True)
```

In [158…
```
df.head(3)
```
Out[158]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	SOYBEAN MEAL	ZINC	ULS DIESEL	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON
2000-01-04	2.176	283.7	25.55	24.39	464.25	203.00	0.8480	5.375	213.50	69.075	...	147.8	1215.00	67.78	8314.0	247.25	5.77	116.25	55.625	274.00	50.73
2000-01-05	2.168	282.1	24.91	23.73	469.25	203.00	0.8565	5.210	213.00	68.975	...	150.7	1208.25	66.55	8307.0	249.75	5.81	118.60	55.975	276.25	51.56
2000-01-06	2.196	282.4	24.78	23.62	468.00	203.75	0.8530	5.167	211.25	70.075	...	149.1	1212.00	66.28	8252.0	248.50	5.77	116.85	55.175	276.00	52.08

3 rows × 22 columns

In [159…
```
df.isna().sum().plot(kind="bar")
```
Out[159]:
```
<Axes: >
```



In [160]:
```
df.dropna(inplace=True)
```

In [161]:
```
df[df.duplicated()]
```
Out[161]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	SOYBEAN MEAL	ZINC	ULS DIESEL	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON
2012-04-06	2.089	1628.5	103.31	123.43	1434.0	658.25	3.7805	31.730	1007.00	116.325	...	391.9	2001.5	316.90	18342.0	638.0	19.94	183.00	84.600	662.0	89.54
2017-01-02	3.724	1151.7	53.72	56.82	996.5	352.00	2.5055	15.889	501.00	116.000	...	312.9	2583.5	170.42	9975.0	408.0	19.51	137.05	66.150	418.5	70.95
2021-04-02	2.639	1728.5	61.45	64.86	1402.5	559.75	3.9805	24.940	496.25	120.025	...	410.2	2706.0	163.10	16172.5	611.0	14.71	121.80	101.775	595.0	77.95

3 rows × 22 columns

In [162]:
```
df.duplicated().sum()
```
Out[162]:
```
3
```

In [163]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6039 entries, 2000-01-04 to 2023-08-03
Data columns (total 22 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6039 non-null   float64
 1   GOLD                 6039 non-null   float64
 2   WTI CRUDE            6039 non-null   float64
 3   BRENT CRUDE          6039 non-null   float64
 4   SOYBEANS             6039 non-null   float64
 5   CORN                 6039 non-null   float64
 6   COPPER               6039 non-null   float64
 7   SILVER               6039 non-null   float64
 8   LOW SULPHUR GAS OIL  6039 non-null   float64
 9   LIVE CATTLE          6039 non-null   float64
 10  SOYBEAN OIL          6039 non-null   float64
 11  ALUMINIUM            6039 non-null   float64
 12  SOYBEAN MEAL         6039 non-null   float64
 13  ZINC                 6039 non-null   float64
 14  ULS DIESEL           6039 non-null   float64
 15  NICKEL               6039 non-null   float64
 16  WHEAT                6039 non-null   float64
 17  SUGAR                6039 non-null   float64
 18  COFFEE               6039 non-null   float64
 19  LEAN HOGS            6039 non-null   float64
 20  HRW WHEAT            6039 non-null   float64
 21  COTTON               6039 non-null   float64
dtypes: float64(22)
memory usage: 1.1 MB
```

In [164]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6039 entries, 2000-01-04 to 2023-08-03
Data columns (total 22 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6039 non-null   float64
 1   GOLD                 6039 non-null   float64
 2   WTI CRUDE            6039 non-null   float64
 3   BRENT CRUDE          6039 non-null   float64
 4   SOYBEANS             6039 non-null   float64
 5   CORN                 6039 non-null   float64
 6   COPPER               6039 non-null   float64
 7   SILVER               6039 non-null   float64
 8   LOW SULPHUR GAS OIL  6039 non-null   float64
 9   LIVE CATTLE          6039 non-null   float64
 10  SOYBEAN OIL          6039 non-null   float64
 11  ALUMINIUM            6039 non-null   float64
 12  SOYBEAN MEAL         6039 non-null   float64
 13  ZINC                 6039 non-null   float64
 14  ULS DIESEL           6039 non-null   float64
 15  NICKEL               6039 non-null   float64
 16  WHEAT                6039 non-null   float64
 17  SUGAR                6039 non-null   float64
 18  COFFEE               6039 non-null   float64
 19  LEAN HOGS            6039 non-null   float64
 20  HRW WHEAT            6039 non-null   float64
 21  COTTON               6039 non-null   float64
dtypes: float64(22)
memory usage: 1.1 MB
```

In [165]:
```
# splitting index into weeks months and year
df.index=pd.to_datetime(df.index)
df["weeks of the year"]=df.index.isocalendar().week
df["months of the year"]=df.index.month
df["year"]=df.index.year
```

In [166]:
```
df.drop(["2012-04-06" ,"2017-01-02","2021-04-02"],axis =0 ,inplace=True)
```

In [167]:
```
df.duplicated().sum()
```
Out[167]:
```
0
```

In [168]:
```
first="2021-04-01"
second="2021-04-02"
df.loc[first : second]
```
Out[168]:

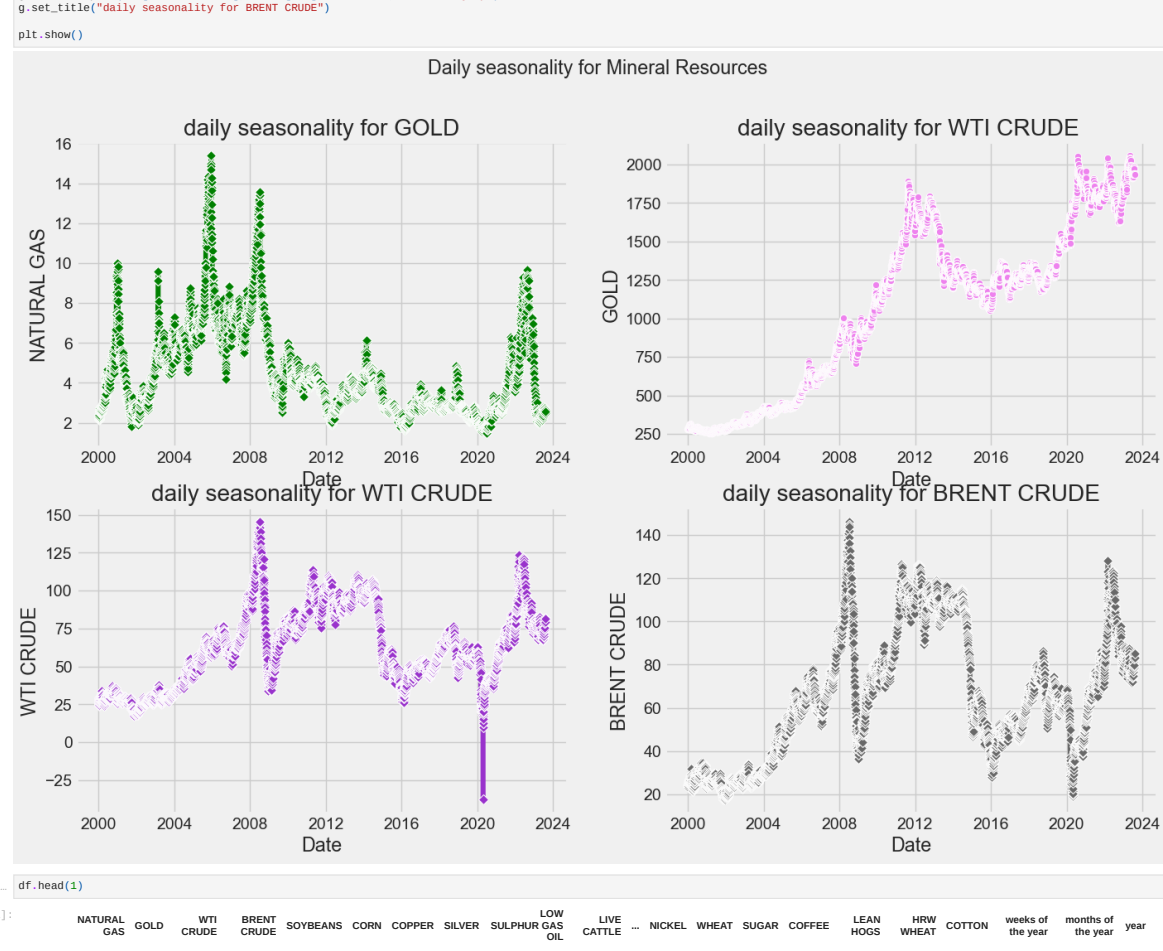
Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON	weeks of the year	months of the year	year
2021-04-01	2.639	1728.5	61.45	64.86	1402.5	559.75	3.9805	24.940	496.25	120.025	...	16172.5	611.0	14.71	121.8	101.775	595.0	77.95	13	4	2021

1 rows × 25 columns

In [169]:
```
df.dropna(inplace=True)
```

data Visualization

In [170]:
```
# plot subplots for daily seasonality in united states
plt.style.use("fivethirtyeight")
fig,axs = plt.subplots (nrows=2, ncols=2, sharey=False ,figsize= (15,10))
fig.suptitle("Daily seasonality for Mineral Resources")
g= sns.lineplot(df["NATURAL GAS"],ax=axs[0,0],marker="o",color ="green")
g.set_title("daily seasonality for GOLD")
g= sns.lineplot(df["GOLD"], ax=axs[0,1], marker="o",color ="violet")
g.set_title("daily seasonality for WTI CRUDE ")
g= sns.lineplot(df["WTI CRUDE"],ax=axs[1,0],marker="o",color="darkorchid")
g.set_title("daily seasonality for WTI CRUDE")
g= sns.lineplot( df["BRENT CRUDE"],ax=axs[1,1],marker="o",color="dimgray")
g.set_title("daily seasonality for BRENT CRUDE")

plt.show()
```



In [171]:
```
df.head(1)
```
Out[171]:

Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON	weeks of the year	months of the year	year
2000-01-04	2.176	283.7	25.55	24.39	464.25	203.0	0.848	5.375	213.5	69.075	...	8314.0	247.25	5.77	116.25	55.625	274.0	50.73	1	1	2000

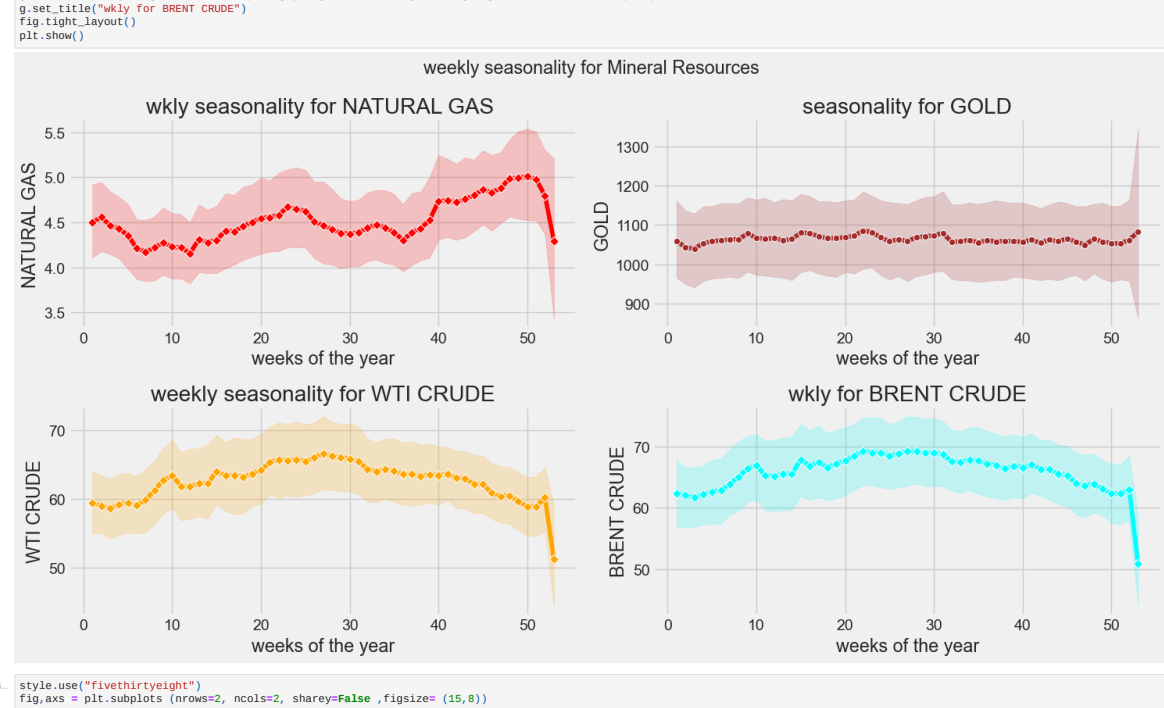
1 rows × 25 columns

In [172]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6036 entries, 2000-01-04 to 2023-08-03
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6036 non-null   float64
 1   GOLD                 6036 non-null   float64
 2   WTI CRUDE            6036 non-null   float64
 3   BRENT CRUDE          6036 non-null   float64
 4   SOYBEANS             6036 non-null   float64
 5   CORN                 6036 non-null   float64
 6   COPPER               6036 non-null   float64
 7   SILVER               6036 non-null   float64
 8   LOW SULPHUR GAS OIL  6036 non-null   float64
 9   LIVE CATTLE          6036 non-null   float64
 10  SOYBEAN OIL          6036 non-null   float64
 11  ALUMINIUM            6036 non-null   float64
 12  SOYBEAN MEAL         6036 non-null   float64
 13  ZINC                 6036 non-null   float64
 14  ULS DIESEL           6036 non-null   float64
 15  NICKEL               6036 non-null   float64
 16  WHEAT                6036 non-null   float64
 17  SUGAR                6036 non-null   float64
 18  COFFEE               6036 non-null   float64
 19  LEAN HOGS            6036 non-null   float64
 20  HRW WHEAT            6036 non-null   float64
 21  COTTON               6036 non-null   float64
 22  weeks of the year    6036 non-null   UInt32
 23  months of the year   6036 non-null   int64
 24  year                 6036 non-null   int64
dtypes: UInt32(1), float64(22), int64(2)
memory usage: 1.3 MB
```

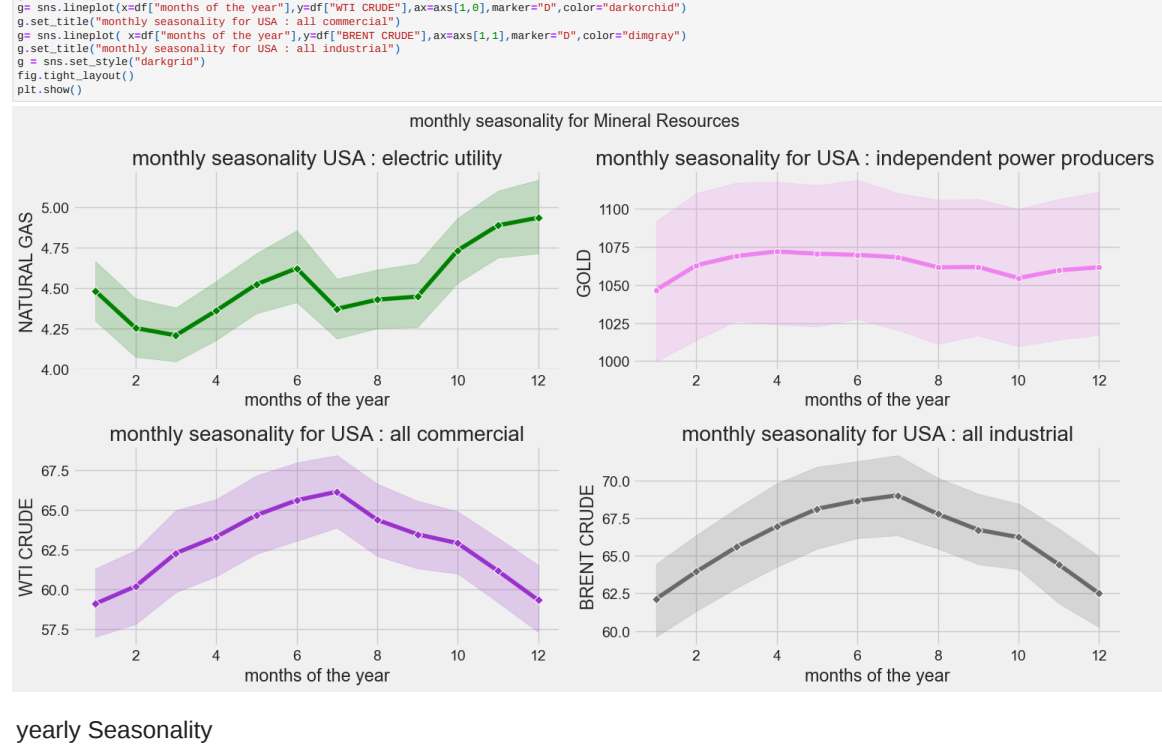
In [173]:
```
df["weeks of the year"]=df["weeks of the year"].values.astype("int32")
```

In [174]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6036 entries, 2000-01-04 to 2023-08-03
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6036 non-null   float64
 1   GOLD                 6036 non-null   float64
 2   WTI CRUDE            6036 non-null   float64
 3   BRENT CRUDE          6036 non-null   float64
 4   SOYBEANS             6036 non-null   float64
 5   CORN                 6036 non-null   float64
 6   COPPER               6036 non-null   float64
 7   SILVER               6036 non-null   float64
 8   LOW SULPHUR GAS OIL  6036 non-null   float64
 9   LIVE CATTLE          6036 non-null   float64
 10  SOYBEAN OIL          6036 non-null   float64
 11  ALUMINIUM            6036 non-null   float64
 12  SOYBEAN MEAL         6036 non-null   float64
 13  ZINC                 6036 non-null   float64
 14  ULS DIESEL           6036 non-null   float64
 15  NICKEL               6036 non-null   float64
 16  WHEAT                6036 non-null   float64
 17  SUGAR                6036 non-null   float64
 18  COFFEE               6036 non-null   float64
 19  LEAN HOGS            6036 non-null   float64
 20  HRW WHEAT            6036 non-null   float64
 21  COTTON               6036 non-null   float64
 22  weeks of the year    6036 non-null   int32
 23  months of the year   6036 non-null   int64
 24  year                 6036 non-null   int64
dtypes: float64(22), int32(1), int64(2)
memory usage: 1.3 MB
```

In [175]:
```
style.use("fivethirtyeight")
fig,axs = plt.subplots (nrows=2, ncols=2, sharey=False ,figsize= (16,8))
fig.suptitle("weekly seasonality for Mineral Resources")
g= sns.lineplot(x=df["weeks of the year"],y=df["NATURAL GAS"],ax=axs[0,0],marker="o",color ="red")
g.set_title("wkly seasonality for NATURAL GAS")
g= sns.lineplot(x=df["weeks of the year"],y=df["GOLD"],ax=axs[0,1],marker="o",color="brown")
g.set_title(" seasonality for GOLD")
g= sns.lineplot(x=df["weeks of the year"],y=df["WTI CRUDE"],ax=axs[1,0],marker="o",color="orange")
g.set_title("weekly seasonality for WTI CRUDE")
g= sns.lineplot(x=df["weeks of the year"],y=df["BRENT CRUDE"],ax=axs[1,1],marker="o",color="cyan")
g.set_title("wkly for BRENT CRUDE")
fig.tight_layout()
plt.show()
```

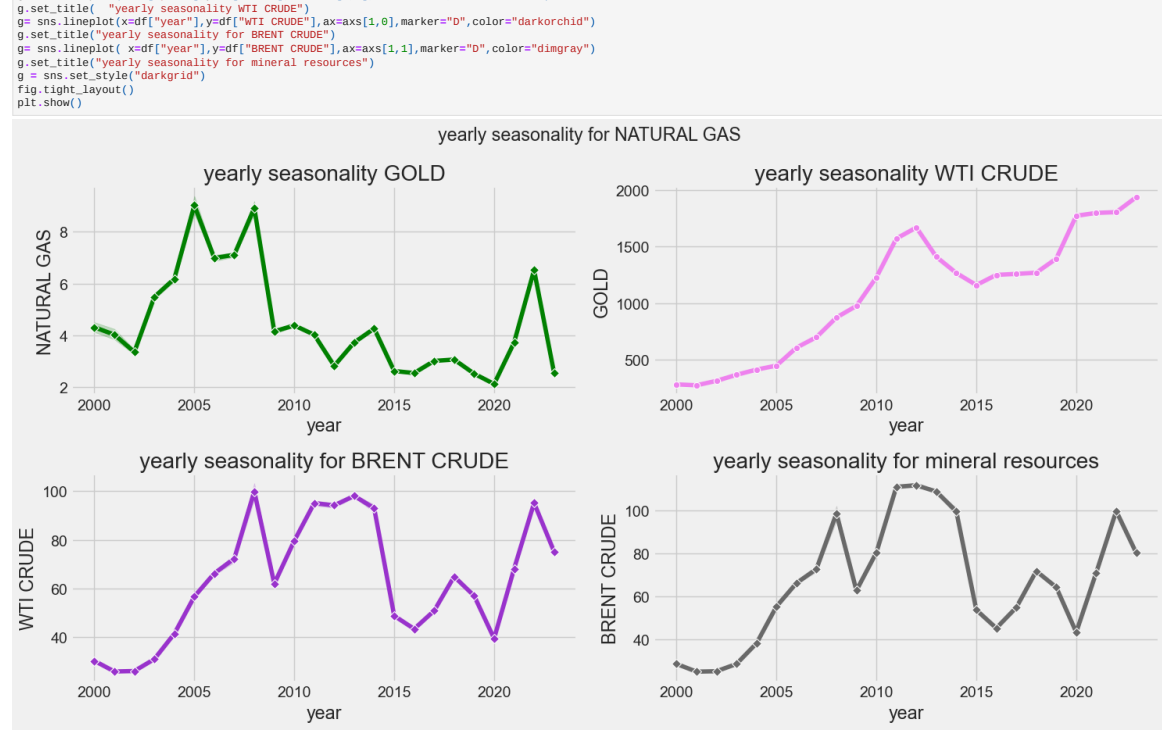


In [176]:
```
style.use("fivethirtyeight")
fig,axs = plt.subplots (nrows=2, ncols=2, sharey=False ,figsize= (16,8))
fig.suptitle("monthly seasonality for Mineral Resources")
g= sns.lineplot(x=df["months of the year"],y=df["NATURAL GAS"],ax=axs[0,0],marker="o",color ="green")
g.set_title("monthly seasonality USA : electric utility")
g= sns.lineplot(x=df["months of the year"],y=df["GOLD"],ax=axs[0,1],marker="o",color ="violet")
g.set_title("monthly seasonality for USA : independent power producers ")
g= sns.lineplot(x=df["months of the year"],y=df["WTI CRUDE"],ax=axs[1,0],marker="o",color="darkorchid")
g.set_title("monthly seasonality for USA : all commercial")
g= sns.lineplot(x=df["months of the year"],y=df["BRENT CRUDE"],ax=axs[1,1],marker="o",color="dimgray")
g.set_title("monthly seasonality for USA : all industrial")
g = sns.set_style("darkgrid")
fig.tight_layout()
plt.show()
```



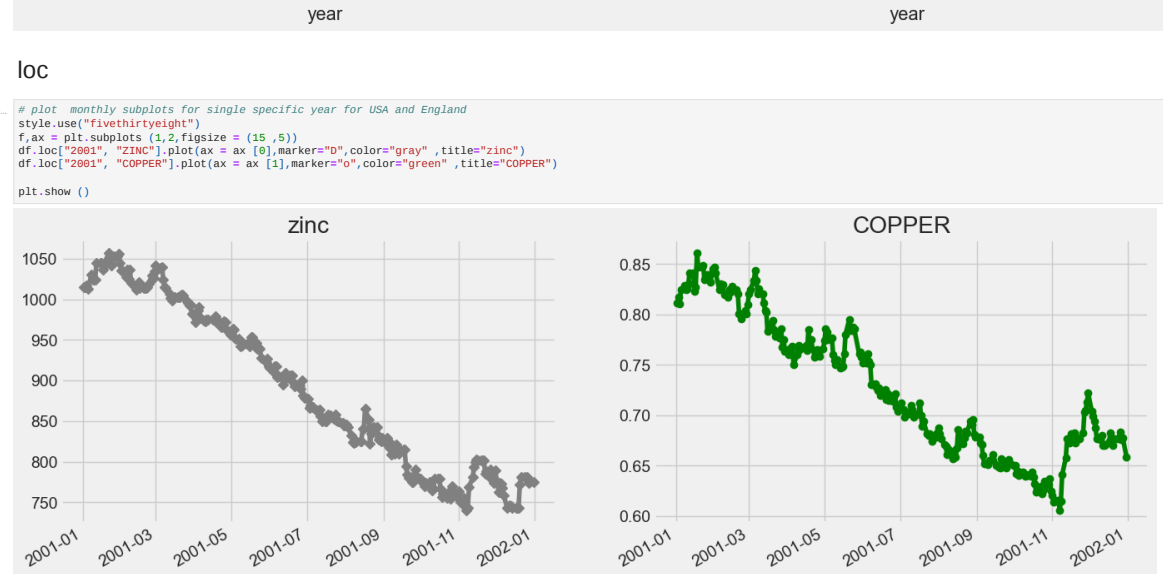
yearly Seasonality

In [177]:
```
style.use("fivethirtyeight")
fig,axs = plt.subplots (nrows=2, ncols=2, sharey=False ,figsize= (16,8))
fig.suptitle("yearly seasonality for NATURAL GAS")
g= sns.lineplot(x=df["year"],y=df["NATURAL GAS"],ax=axs[0,0],marker="o",color ="green")
g.set_title("yearly seasonality GOLD")
g= sns.lineplot(x=df["year"],y=df["GOLD"],ax=axs[0,1], marker="o",color ="violet")
g.set_title(" yearly seasonality WTI CRUDE ")
g= sns.lineplot(x=df["year"],y=df["WTI CRUDE"],ax=axs[1,0],marker="o",color="darkorchid")
g.set_title("yearly seasonality for BRENT CRUDE")
g= sns.lineplot(x=df["year"],y=df["BRENT CRUDE"],ax=axs[1,1],marker="o",color="dimgray")
g.set_title("yearly seasonality for mineral resources")
g = sns.set_style("darkgrid")
fig.tight_layout()
plt.show()
```



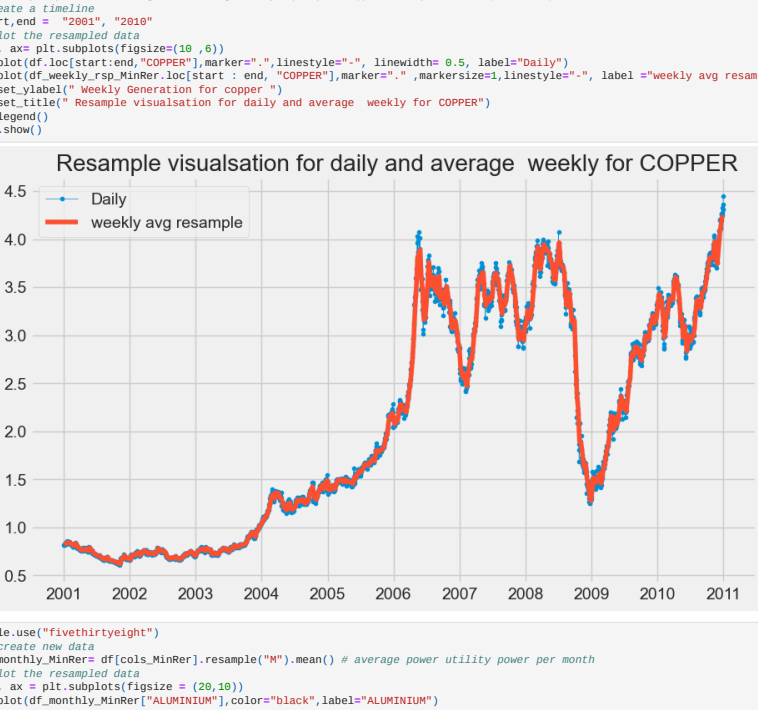
loc

In [178]:
```
# plot  monthly subplots for single specific year for USA and England
style.use("fivethirtyeight")
f,ax = plt.subplots (1,2,figsize = (10 ,4))
df.loc["2001", "ZINC"].plot(ax = ax [0],marker="o",color ="gray" ,title="zinc")
df.loc["2001", "COPPER"].plot(ax = ax [1],marker="o",color="green" ,title="COPPER")
plt.show ()
```

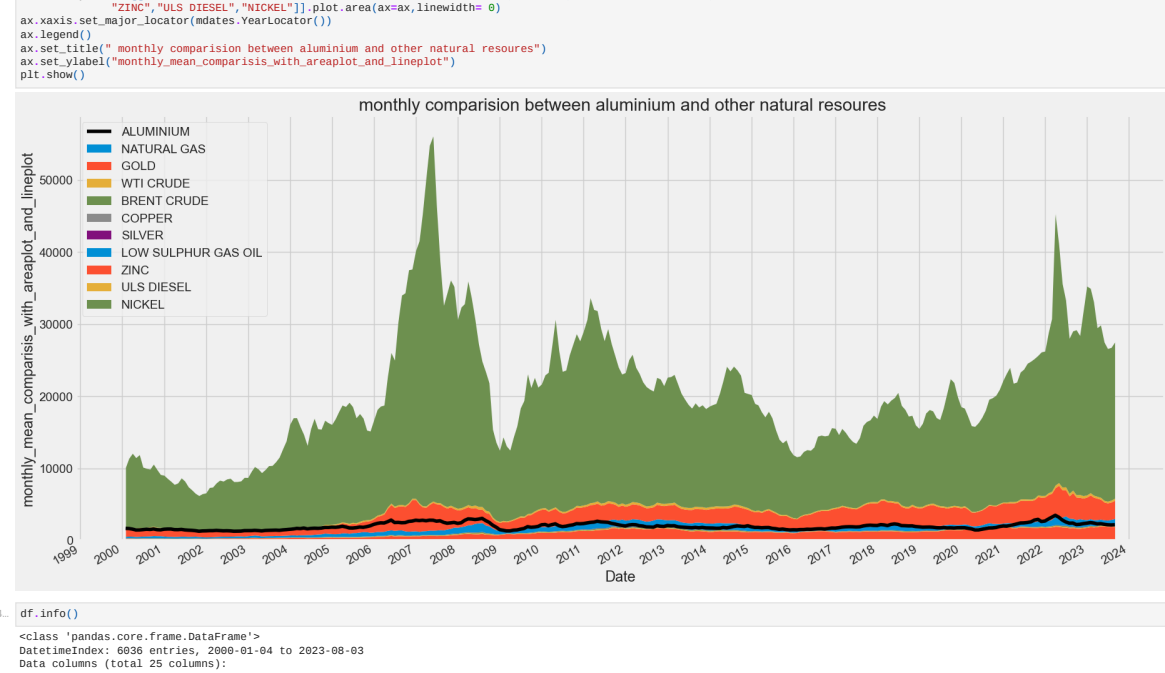


In [179]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6036 entries, 2000-01-04 to 2023-08-03
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6036 non-null   float64
 1   GOLD                 6036 non-null   float64
 2   WTI CRUDE            6036 non-null   float64
 3   BRENT CRUDE          6036 non-null   float64
 4   SOYBEANS             6036 non-null   float64
 5   CORN                 6036 non-null   float64
 6   COPPER               6036 non-null   float64
 7   SILVER               6036 non-null   float64
 8   LOW SULPHUR GAS OIL  6036 non-null   float64
 9   LIVE CATTLE          6036 non-null   float64
 10  SOYBEAN OIL          6036 non-null   float64
 11  ALUMINIUM            6036 non-null   float64
 12  SOYBEAN MEAL         6036 non-null   float64
 13  ZINC                 6036 non-null   float64
 14  ULS DIESEL           6036 non-null   float64
 15  NICKEL               6036 non-null   float64
 16  WHEAT                6036 non-null   float64
 17  SUGAR                6036 non-null   float64
 18  COFFEE               6036 non-null   float64
 19  LEAN HOGS            6036 non-null   float64
 20  HRW WHEAT            6036 non-null   float64
 21  COTTON               6036 non-null   float64
 22  weeks of the year    6036 non-null   int32
 23  months of the year   6036 non-null   int64
 24  year                 6036 non-null   int64
dtypes: float64(22), int32(1), int64(2)
memory usage: 1.3 MB
```

In [180]:
```
# plot style
style.use("fivethirtyeight")
# create new data
cols_MinRer = ["NATURAL GAS","GOLD","WTI CRUDE","BRENT CRUDE",
               "COPPER" ,"SILVER","LOW SULPHUR GAS OIL","ALUMINIUM",
               "ZINC","ULS DIESEL","NICKEL"]
# resample data to weekly frequency
df_weekly_rsp_MinRer = df[cols_MinRer].resample("W").mean() # average mineral produce  per week
#create a timeline
start,end = "2001", "2010"
# plot the resampled data
fig, ax= plt.subplots(figsize= (10 ,6))
ax.plot(df.loc[start:end, "COPPER"],marker=".", linestyle="-", linewidth= 0.5, label="Daily")
ax.plot(df_weekly_rsp_MinRer.loc[start : end, "COPPER"],marker="o",markersize=1,linestyle="-", label ="weekly avg resample")
ax.set_ylabel(" Weekly Generation for copper ")
ax.set_title(" Resample visualsation for daily and average  weekly for COPPER")
ax.legend()
plt.show()
```



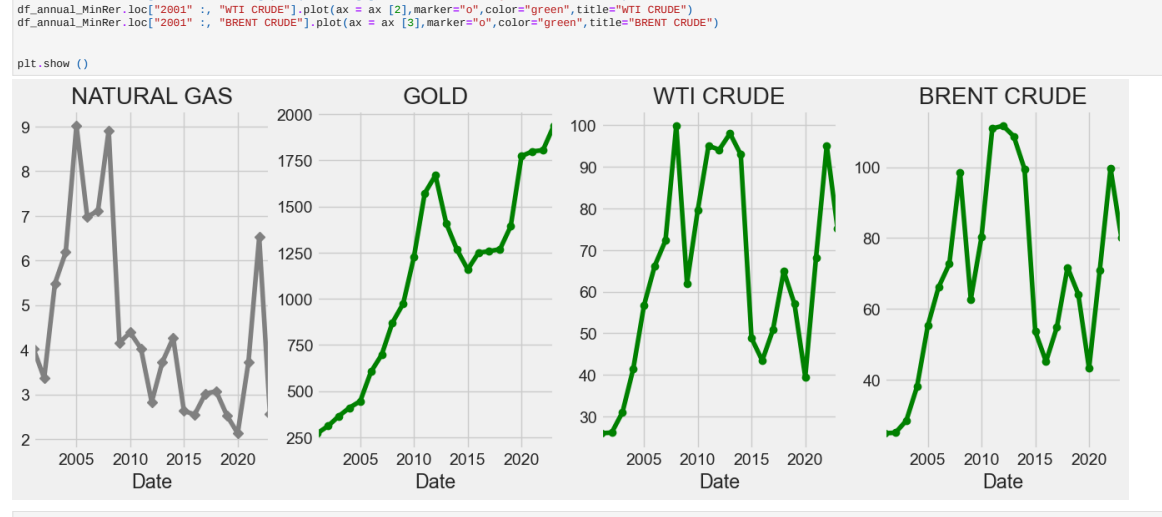
In [181]:
```
style.use("fivethirtyeight")
# create new data
df_monthly_MinRer= df[cols_MinRer].resample("M").mean() # average power utility power per month
# plot the resampled data
fig, ax= plt.subplots(figsize = (20,10))
ax.plot(df_monthly_MinRer["ALUMINIUM"],color="black",label="ALUMINIUM")
df_monthly_MinRer[["NATURAL GAS","GOLD","WTI CRUDE","BRENT CRUDE","COPPER" ,"SILVER","LOW SULPHUR GAS OIL",
              "ZINC","ULS DIESEL","NICKEL"]].plot.area(ax=ax,linewidth=0)
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.legend()
ax.set_title( "monthly comparision between aluminium and other natural resources")
ax.set_ylabel("monthly_mean_comparisin_with_areaplot_and_lineplot")
plt.show()
```



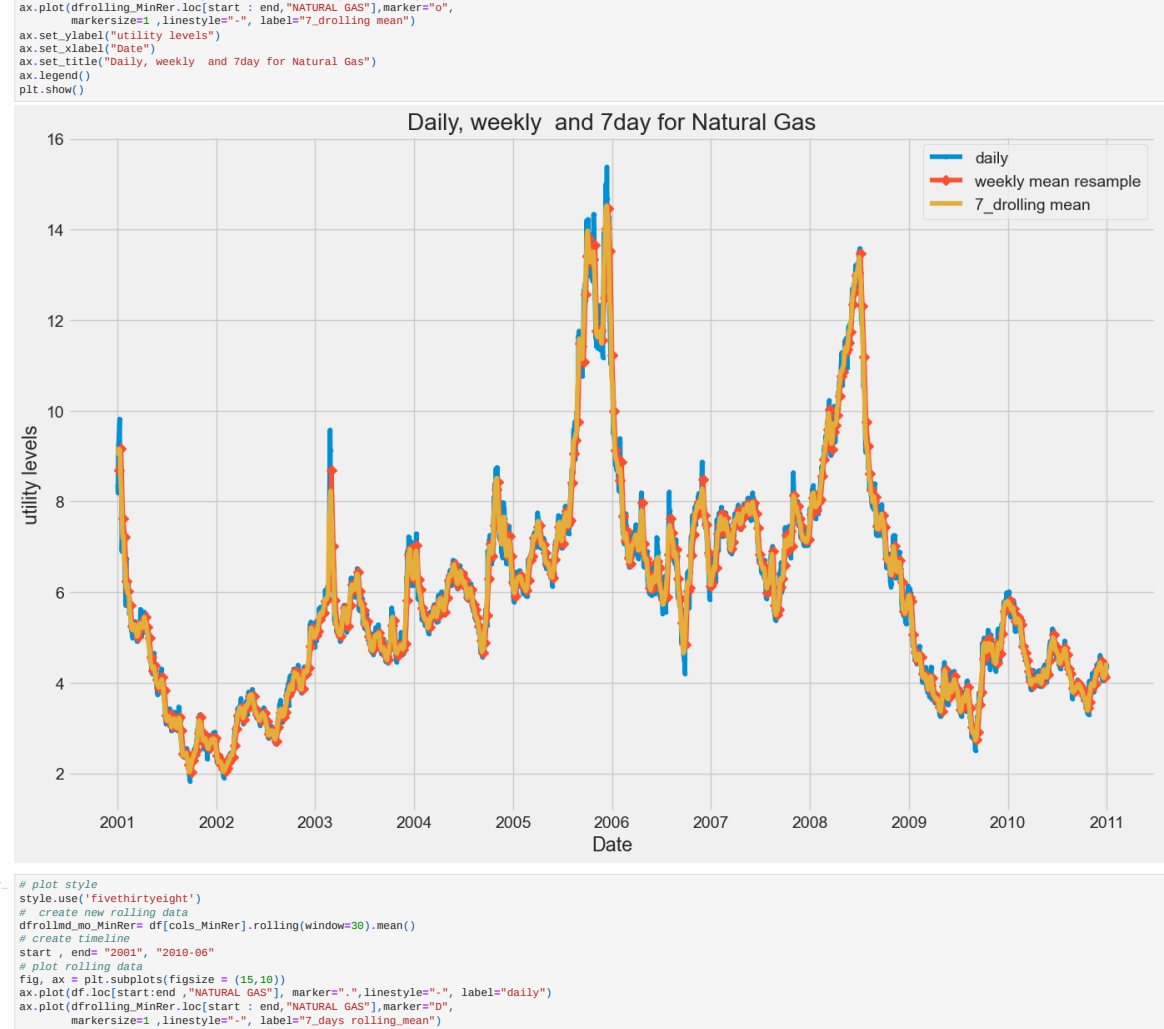
In [182]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6036 entries, 2000-01-04 to 2023-08-03
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6036 non-null   float64
 1   GOLD                 6036 non-null   float64
 2   WTI CRUDE            6036 non-null   float64
 3   BRENT CRUDE          6036 non-null   float64
 4   SOYBEANS             6036 non-null   float64
 5   CORN                 6036 non-null   float64
 6   COPPER               6036 non-null   float64
 7   SILVER               6036 non-null   float64
 8   LOW SULPHUR GAS OIL  6036 non-null   float64
 9   LIVE CATTLE          6036 non-null   float64
 10  SOYBEAN OIL          6036 non-null   float64
 11  ALUMINIUM            6036 non-null   float64
 12  SOYBEAN MEAL         6036 non-null   float64
 13  ZINC                 6036 non-null   float64
 14  ULS DIESEL           6036 non-null   float64
 15  NICKEL               6036 non-null   float64
 16  WHEAT                6036 non-null   float64
 17  SUGAR                6036 non-null   float64
 18  COFFEE               6036 non-null   float64
 19  LEAN HOGS            6036 non-null   float64
 20  HRW WHEAT            6036 non-null   float64
 21  COTTON               6036 non-null   float64
 22  weeks of the year    6036 non-null   int32
 23  months of the year   6036 non-null   int64
 24  year                 6036 non-null   int64
dtypes: float64(22), int32(1), int64(2)
memory usage: 1.3 MB
```

Annuaul resample of in intervals of 5 years

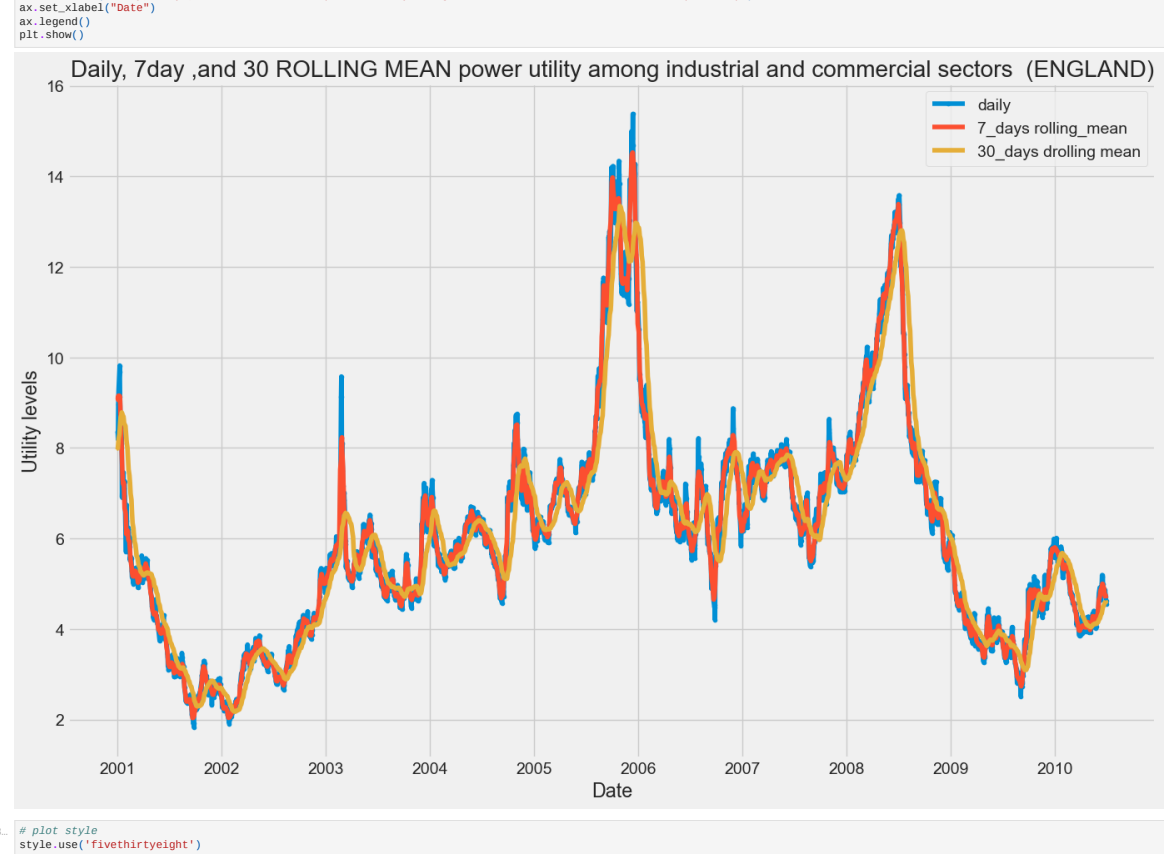
In [183]:
```
# plot style
style.use("fivethirtyeight")
# create new data
df_annual_MinRer= df[cols_MinRer].resample("A").mean() # average mineral resource generation per year in interval of 5years
# plot the resampled data
f,ax = plt.subplots(1,4 ,figsize = (15 ,3))
df_annual_MinRer.loc["2001" : "2024"]["NATURAL GAS"].plot(ax = ax [0],marker="o",color="gray",title="NATURAL GAS")
df_annual_MinRer.loc["2001" : "2024"]["GOLD"].plot(ax = ax [1],marker="o",color="green",title="GOLD")
df_annual_MinRer.loc["2001" : "2024"]["WTI CRUDE"].plot(ax = ax [2],marker="o",color="green",title="WTI CRUDE")
df_annual_MinRer.loc["2001" : "2024"]["BRENT CRUDE"].plot(ax = ax [3],marker="o",color="green",title="BRENT CRUDE")

plt.show()
```



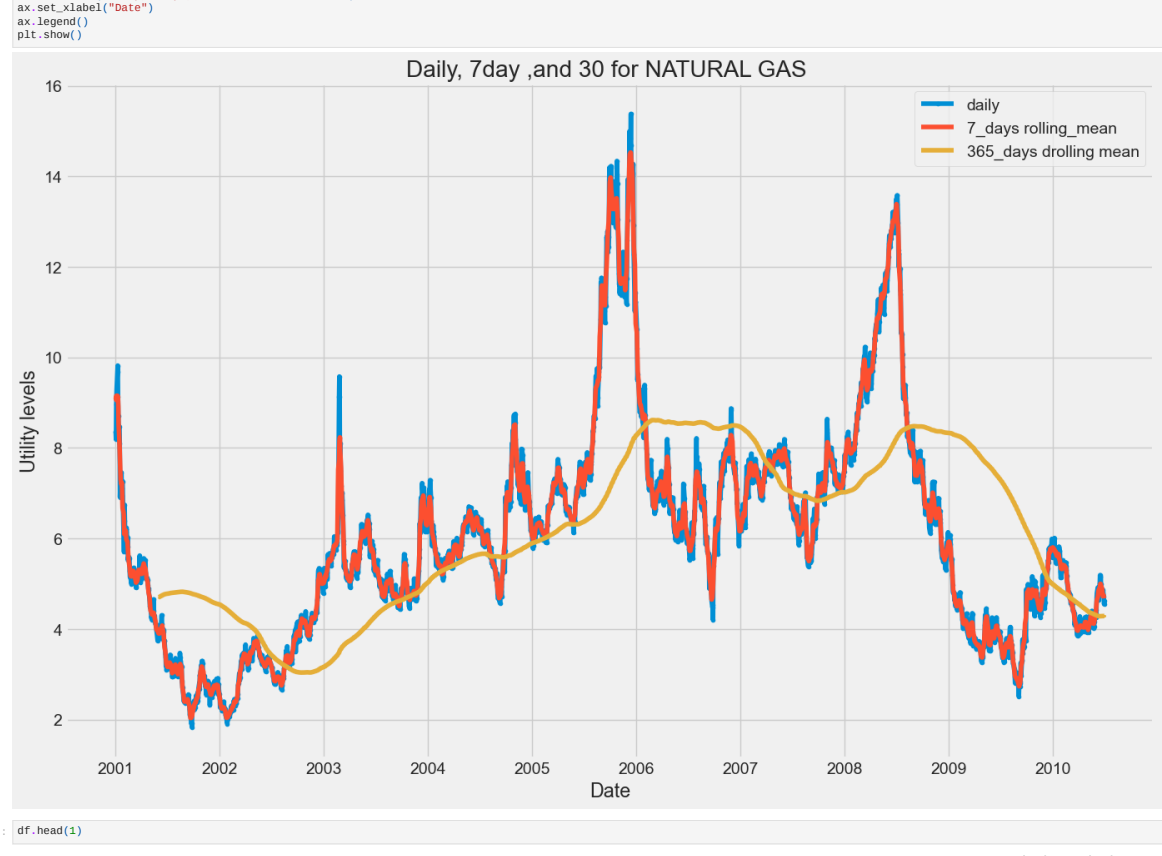
In [184]:
```
# plot style
#style.use("fivethirtyeight")
# create new rolling data
df7rolling_mo_MinRer= df[cols_MinRer].rolling(7, center = True).mean()
# create timeline
start , end = "2001", "2010"
# plot rolling data
fig, ax = plt.subplots(figsize = (15,10))
ax.plot(df.loc[start:end ,"NATURAL GAS"], marker=".", linestyle="-", label="daily")
ax.plot(df_weekly_rsp_MinRer.loc[start : end,"NATURAL GAS"],marker="o",
        markersize=1 ,linestyle="-", label="weekly mean resample")
ax.plot(df7rolling_mo_MinRer.loc[start : end,"NATURAL GAS"],marker="o",
        markersize=1 ,linestyle="-", label="7_drolling_mean")
ax.set_ylabel("utility levels")
ax.set_xlabel("Date")
ax.set_title("Daily, weekly  and 7day for Natural Gas")
ax.legend()
plt.show()
```



In [147]:
```
# plot style
style.use("fivethirtyeight")
#  create new rolling data
dfroll30_mo_MinRer= df[cols_MinRer].rolling(window=30).mean()
# create timeline
start , end= "2001", "2010-06"
# plot rolling data
fig, ax = plt.subplots(figsize = (15,10))
ax.plot(df.loc[start:end ,"NATURAL GAS"], marker=".", linestyle="-", label="daily")
ax.plot(df7rolling_mo_MinRer.loc[start : end,"NATURAL GAS"],marker="o",
        markersize=1 ,linestyle="-", label="7_days rolling_mean")
ax.plot(dfroll30_mo_MinRer.loc[start : end,"NATURAL GAS"],marker="o",
        markersize=1 ,linestyle="-", label="30_days drolling mean")
ax.set_ylabel("Utility levels")
ax.set_title("Daily, 7day ,and 30 ROLLING MEAN power utility among industrial and commercial sectors  (ENGLAND)")
ax.set_xlabel("Date")
ax.legend()
plt.show()
```



In [148]:
```
# plot style
style.use('fivethirtyeight')
#  create new rolling data
dfrollmd_mo_MinRer= df[cols_MinRer].rolling(window=365).mean()
# create timeline
start , end= "2001", "2010-06"
# plot rolling data
fig, ax = plt.subplots(figsize = (15,10))
ax.plot(df.loc[start:end ,"NATURAL GAS"], marker=".",linestyle="-", label="daily")
ax.plot(dfrolling_mo_MinRer.loc[start : end,"NATURAL GAS"],marker="D",
        markersize=1 ,linestyle="-", label="7_days rolling_mean")
ax.plot(dfrollmd_mo_MinRer.loc[start : end,"NATURAL GAS"],marker="o",
        markersize=1 ,linestyle="-", label="365_days drolling mean")
ax.set_ylabel("Utility levels")
ax.set_title("Daily, 7day ,and 30 for NATURAL GAS ")
ax.set_xlabel("Date")
ax.legend()
plt.show()
```



In [71]:
```
df.head(1)
```
Out[71]:

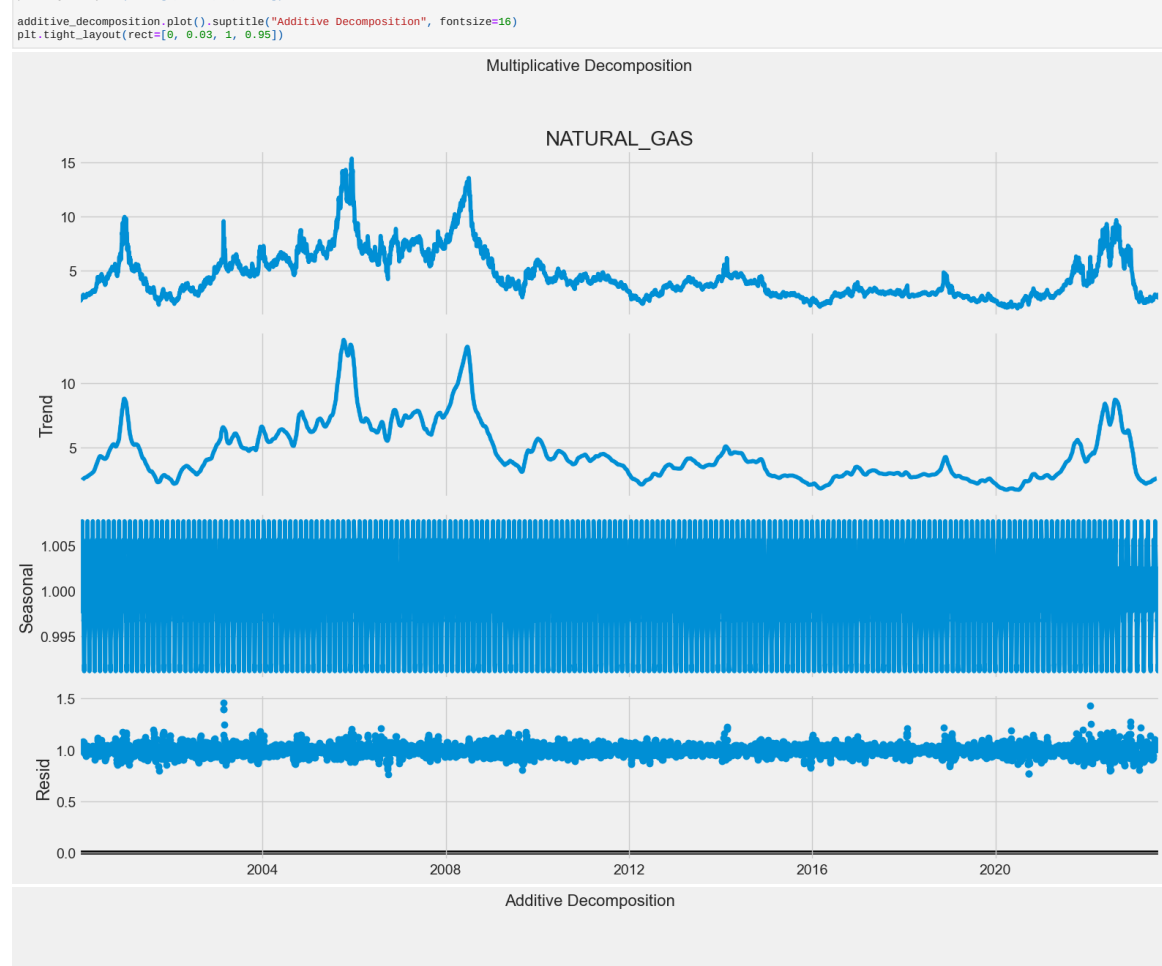
Date	NATURAL GAS	GOLD	WTI CRUDE	BRENT CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW SULPHUR GAS OIL	LIVE CATTLE	...	NICKEL	WHEAT	SUGAR	COFFEE	LEAN HOGS	HRW WHEAT	COTTON	weeks of the year	months of the year	year
2000-01-04	2.176	283.7	25.55	24.39	464.25	203.0	0.848	5.375	213.5	69.075	...	8314.0	247.25	5.77	116.25	55.625	274.0	50.73	1	1	2000

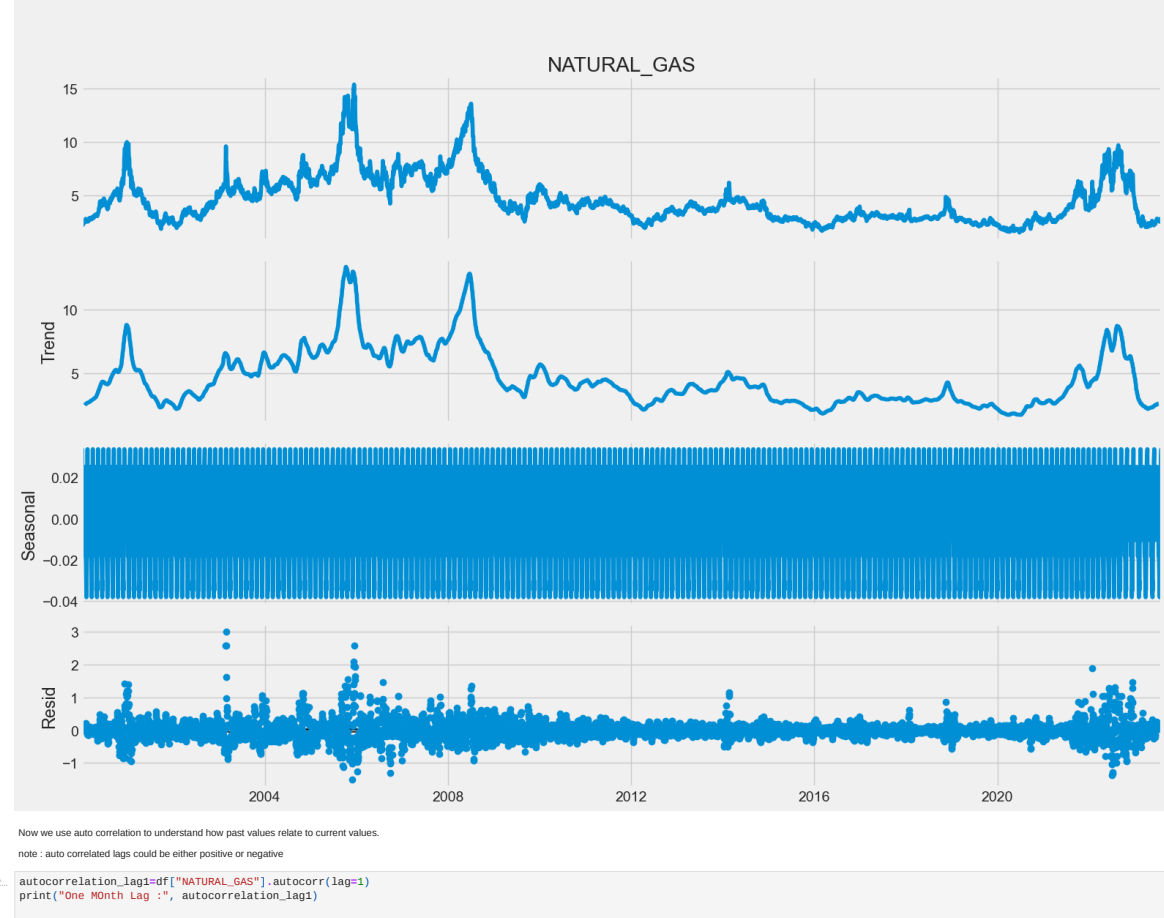
1 rows × 25 columns

In [72]:
```
df.info()
```
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 6036 entries, 2000-01-04 to 2023-08-03
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   NATURAL GAS          6036 non-null   float64
 1   GOLD                 6036 non-null   float64
 2   WTI CRUDE            6036 non-null   float64
 3   BRENT CRUDE          6036 non-null   float64
 4   SOYBEANS             6036 non-null   float64
 5   CORN                 6036 non-null   float64
 6   COPPER               6036 non-null   float64
 7   SILVER               6036 non-null   float64
 8   LOW SULPHUR GAS OIL  6036 non-null   float64
 9   LIVE CATTLE          6036 non-null   float64
 10  SOYBEAN OIL          6036 non-null   float64
 11  ALUMINIUM            6036 non-null   float64
 12  SOYBEAN MEAL         6036 non-null   float64
 13  ZINC                 6036 non-null   float64
 14  ULS DIESEL           6036 non-null   float64
 15  NICKEL               6036 non-null   float64
 16  WHEAT                6036 non-null   float64
 17  SUGAR                6036 non-null   float64
 18  COFFEE               6036 non-null   float64
 19  LEAN HOGS            6036 non-null   float64
 20  HRW WHEAT            6036 non-null   float64
 21  COTTON               6036 non-null   float64
 22  weeks of the year    6036 non-null   int32
 23  months of the year   6036 non-null   int64
 24  year                 6036 non-null   int64
dtypes: float64(22), int32(1), int64(2)
memory usage: 1.3 MB
```

In [180]:
```
# rename column names of dataset
df.rename(columns={df.columns[0] : "NATURAL_GAS" , df.columns[2] : "WTI_CRUDE" ,
                df.columns[9] : "LIVE_CATTLE" ,df.columns[10] :"SOYBEAN_OIL",
                df.columns[8] :"LOW_SULPHUR_GAS_OIL",
                df.columns[12]: "SOYBEAN_MEAL",df.columns[20] :"HRW_WHEAT",
                df.columns[3] :"BRENT_CRUDE",
                df.columns[19] :"LEAN_HOGS",
                df.columns[22] :"weeks_of_the_year",
                df.columns[23] : "months_of_the_year"},inplace= True)
```

In [181]:
```
style.use("fivethirtyeight")
from statsmodels.tsa.seasonal import seasonal_decompose


# Multiplicative Decomposition
multiplicative_decomposition = seasonal_decompose(df["NATURAL_GAS"], model="multiplicative", period=30)

# Additive Decomposition
additive_decomposition = seasonal_decompose(df["NATURAL_GAS"], model="additive", period=30)

# Plot
plt.rcParams.update({"figure.figsize": (16,12)})
multiplicative_decomposition.plot().suptitle("Multiplicative Decomposition", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

additive_decomposition.plot().suptitle("Additive Decomposition", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
```





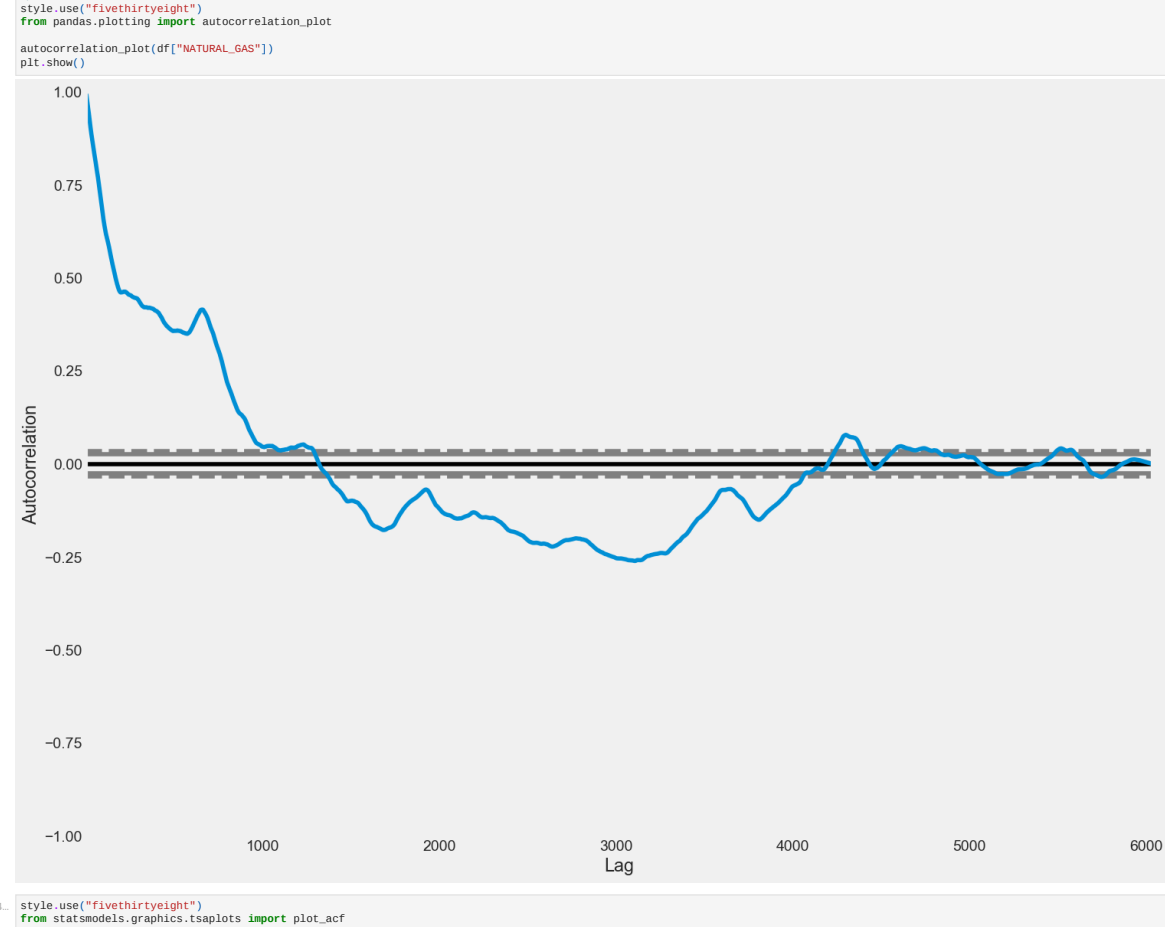
Now we use auto correlation to understand how past values relate to current values.

note : auto correlated lags could be either positive or negative

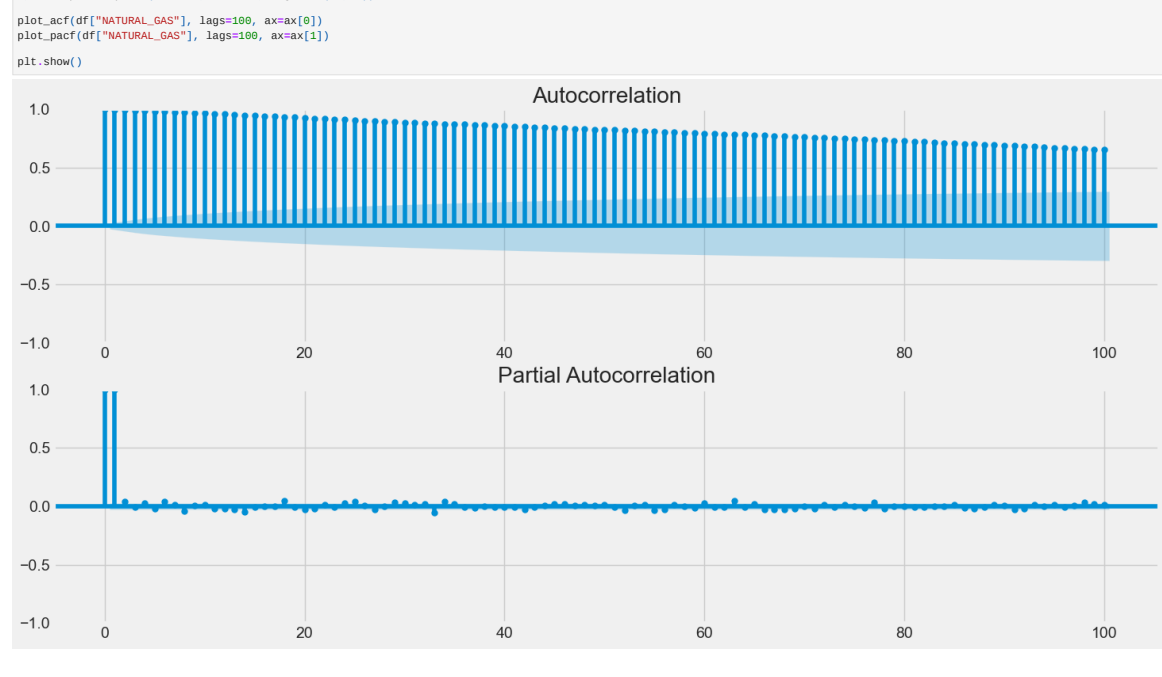
In [182]:
```
autocorrelation_lag1=df["NATURAL_GAS"].autocorr(lag=1)
print("One Month Lag :", autocorrelation_lag1)

autocorrelation_lag2=df["NATURAL_GAS"].autocorr(lag=2)
print("two Month Lag :", autocorrelation_lag2)

autocorrelation_lag3=df["NATURAL_GAS"].autocorr(lag=3)
print("Three Month Lag :", autocorrelation_lag3)

autocorrelation_lag4=df["NATURAL_GAS"].autocorr(lag=4)
print("four Month Lag :", autocorrelation_lag4)
```
```
One Month Lag : 0.996[illegible]
two Month Lag : 0.992[illegible]
Three Month Lag : 0.989[illegible]
four Month Lag : 0.986[illegible]
```

we can see that we have positive auto-correlation for New_England_all_sectors suggesting that high value in the current time period would be a high tomorrow as well

In [183]:
```
#plot autocorrelation
style.use("fivethirtyeight")
from pandas.plotting import autocorrelation_plot

autocorrelation_plot(df["NATURAL_GAS"])
plt.show()
```



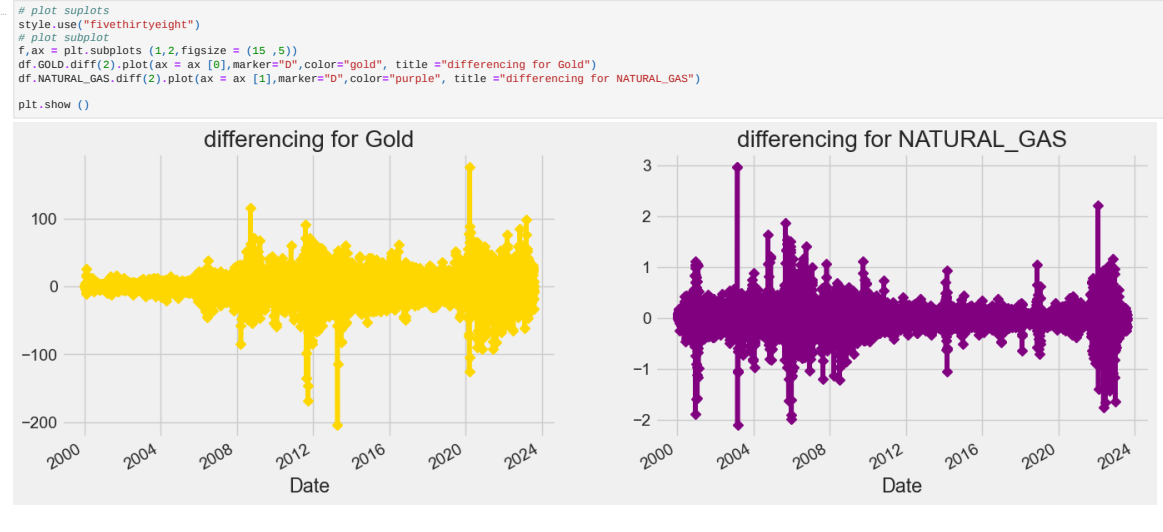
In [184]:
```
style.use("fivethirtyeight")
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.graphics.tsaplots import plot_pacf

f, ax = plt.subplots(nrows=2, ncols=1, figsize=(16, 8))

plot_acf(df["NATURAL_GAS"], lags=100, ax=ax[0])
plot_pacf(df["NATURAL_GAS"], lags=100, ax=ax[1])

plt.show()
```



differencing

The diff function helps remove trends from the data to view their stationarity.

In [185]:
```
# plot supplots
style.use("fivethirtyeight")
# plot subplot
f,ax = plt.subplots (1,2,figsize = (16 ,5))
df.GOLD.diff(1).plot(ax = ax [0],marker="o",color="gold", title ="differencing for Gold")
df.NATURAL_GAS.diff(1).plot(ax = ax [1],marker="o",color="purple", title ="differencing for NATURAL_GAS")

plt.show ()
```

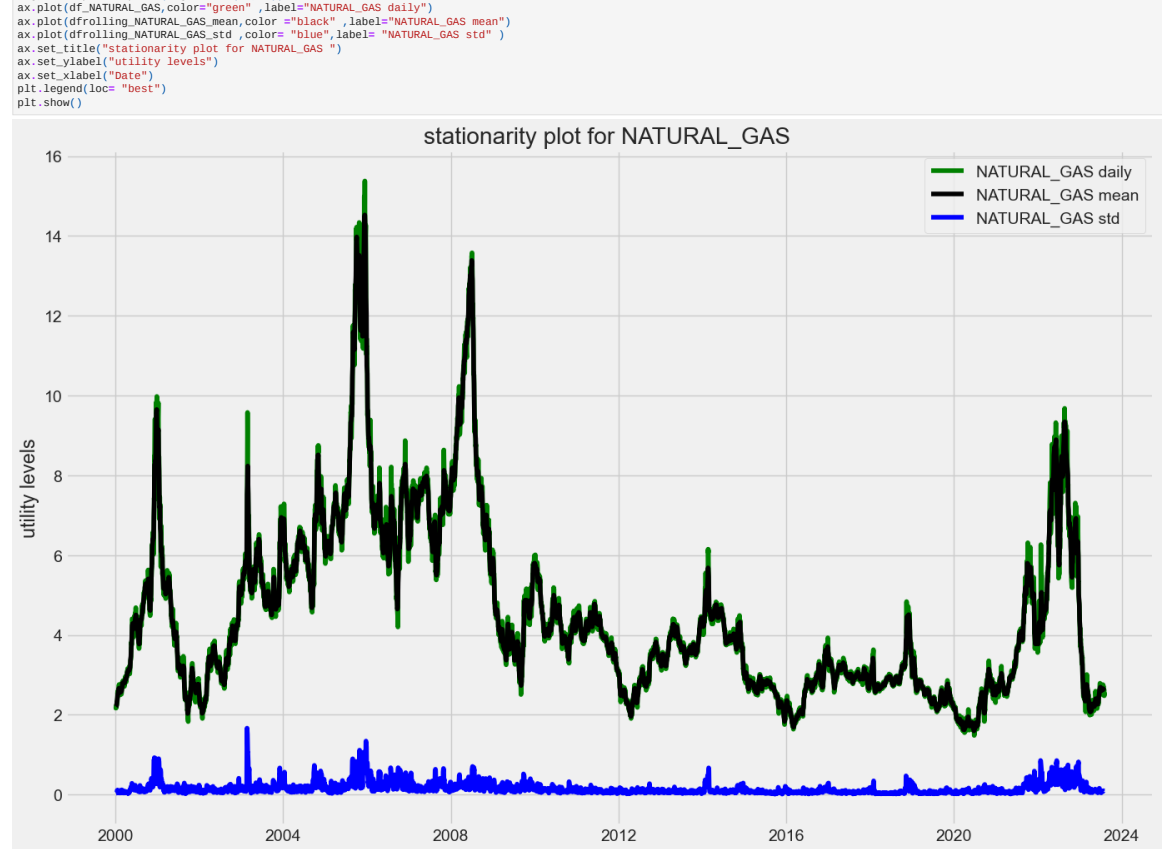


stationaerity

For forecasting purposes, it is desirable for a time series to exhibit stationarity. In a stationary time series, the mean and standard deviation remain constant over time. Stationarity is important because it allows us to make reliable predictions based on the assumption that the future behavior of the time series will be similar to its past behavior.

In [79]:
```
# preparing stationary data
df_NATURAL_GAS=df.NATURAL_GAS
dfrolling_NATURAL_GAS_mean= df.NATURAL_GAS.rolling(7, center = True).mean()
dfrolling_NATURAL_GAS_std= df.NATURAL_GAS.rolling(7, center = True).std()
```

In [80]:
```
# plotting stationary data
fig, ax = plt.subplots(figsize = (15,10))
ax.plot(df_NATURAL_GAS,color="green",label="NATURAL_GAS daily")
ax.plot(dfrolling_NATURAL_GAS_mean,color="black",label="NATURAL_GAS mean")
ax.plot(dfrolling_NATURAL_GAS_std ,color="blue",label="NATURAL_GAS std" )
ax.set_title("stationarity plot for NATURAL_GAS")
ax.set_ylabel("utility levels")
ax.set_xlabel("Date")
plt.legend(loc="best")
plt.show()
```



There are several statistical tests available to determine the stationarity of a time series data. Here we are going to use Augmented Dickey-Fuller (ADF) Test. The ADF tests the null hypothesis that a unit root is present in the time series (indicating non-stationarity).

If the p-value obtained from the ADF test is below a chosen significance level (e.g., 0.05), we can reject the null hypothesis and conclude that the series is stationary.

In [81]:
```
from statsmodels.tsa.stattools import adfuller
result=adfuller(df.NATURAL_GAS)
adf_statistic = result[0]
p_value=result[1]
critical_values= result[4]

print("ADF Statistic:" , adf_statistic)
print("p_value :",p_value)
print ("Critical Values:")
for key, value in critical_values.items():
    print(f"{key}: {value}")
```
```
ADF Statistic: -3.0[illegible]
p_value : 0.03[illegible]
Critical Values:
1%: -3.43[illegible]
5%: -2.86[illegible]
10%: -2.56[illegible]
```

time series forcasting with prophet

In [81]:
```
# pip install prophet
```

In [82]:
```
import prophet
```

In [83]:
```
# import necessary libraries
from prophet import Prophet
```

In [84]:
```
df_model= df.reset_index()
```

In [85]:
```
df_model.head(1)
```
Out[85]:

	Date	NATURAL_GAS	GOLD	WTI_CRUDE	BRENT_CRUDE	SOYBEANS	CORN	COPPER	SILVER	LOW_SULPHUR_GAS_OIL	...	NICKEL	WHEAT	SUGAR	COFFEE	LEAN_HOGS	HRW_WHEAT	COTTON	weeks_of_the_
0	2000-01-04	2.176	283.7	25.55	24.39	464.25	203.0	0.848	5.375	213.5	...	8314.0	247.25	5.77	116.25	55.625	274.0	50.73	1

1 rows × 26 columns

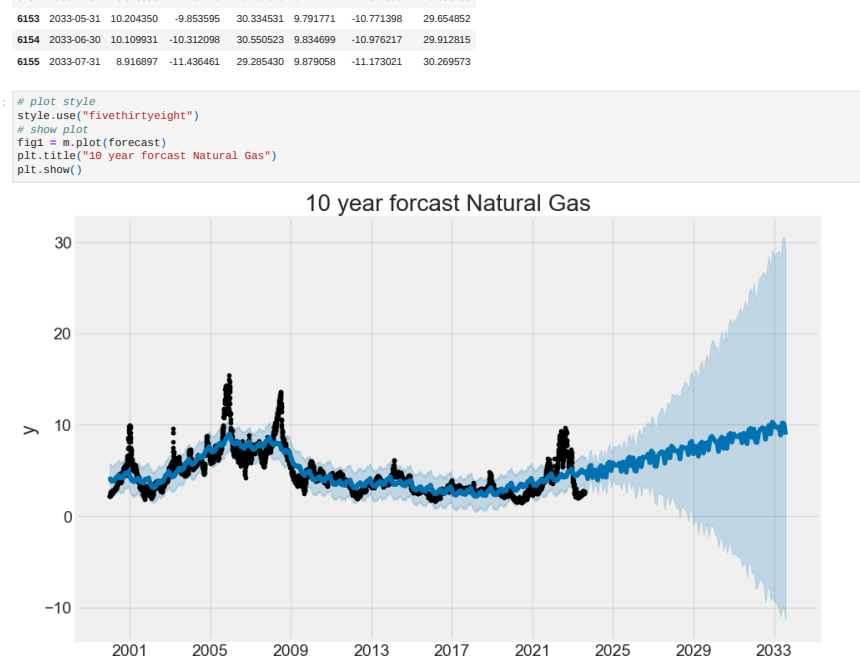
In [86]:
```
df_model["ds"] = pd.to_datetime(df_model["Date"])
df_model["y"] = df_model["NATURAL_GAS"]
```

In [87]:
```
m = Prophet()
m.fit(df_model)
future = m.make_future_dataframe(periods=52 * 10,
                        freq="w")
```
```
22:06:08 - cmdstanpy - INFO - Chain [1] start processing
22:06:10 - cmdstanpy - INFO - Chain [1] done processing
```

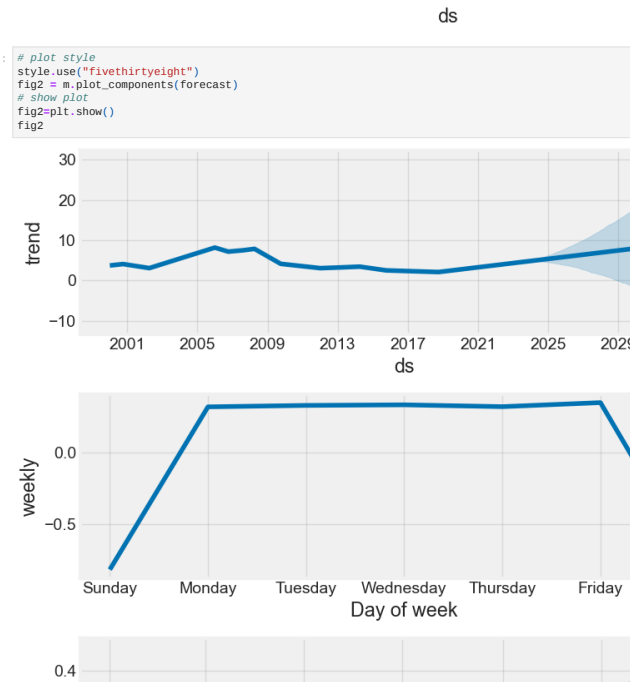
In [88]:
```
# creating future forcasting
forecast = m.predict(future)
forecast[["ds", "yhat", "yhat_lower",
          "yhat_upper", "trend",
          "trend_lower", "trend_upper"]].tail()
```
Out[88]:

	ds	yhat	yhat_lower	yhat_upper	trend	trend_lower	trend_upper
6551	2033-07-03	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
6552	2033-07-10	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
6553	2033-07-17	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
6554	2033-07-24	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]
6555	2033-07-31	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]

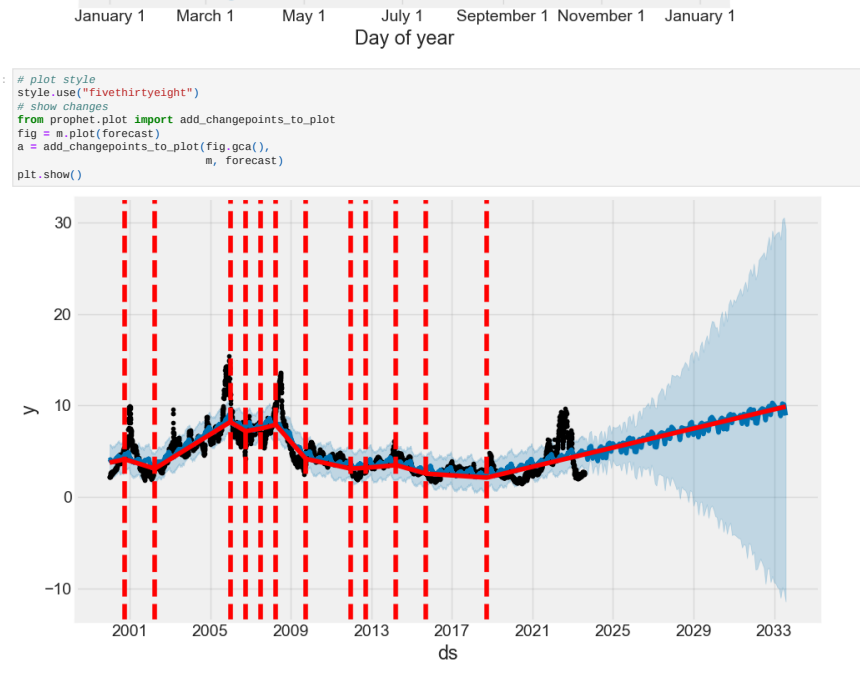
In [89]:
```
# plot style
style.use("fivethirtyeight")
# show plot
fig1 = m.plot(forecast)
plt.title("10 year forcast Natural Gas")
plt.show()
```



```
[illegible]
```



```
[illegible]
```



Next, we split the dataset into training and testing in the ratio 80:20. Only the first column of the data is chosen using iloc[:,:1] and the train_df contains the first training_df_len rows of the original data. The test_df contains all of the remaining rows of the original data starting from training_df_len to the end.

Time series prediction with lstm

In [93]:
```
import math
```

In [94]:
```
# Setting 80 percent data for training
training_df_len = math.ceil(len(df) * .8)
training_df_len

#Splitting the dataset
train_df = df[:training_df_len].iloc[:,:1]
test_df = df[training_df_len:].iloc[:,:1]
print(train_df.shape, test_df.shape)
```
```
(4829, 1) (1207, 1)
```

This code creates a numpy array called dataset_train and populates it with the "NATURAL_GAS" pricing values from the training data. The 1-dimensional array is then transformed into a 2-dimensional array. The shape property, which returns the tuple (num_rows, num_columns) denoting the dataset_train array's final shape

In [95]:
```
# Selecting NATURAL_GAS values
dataset_train = train_df.NATURAL_GAS.values
# Reshaping 1D to 2D array
dataset_train = np.reshape(dataset_train, (-1,1))
dataset_train.shape
```
Out[95]:
```
(4829, 1)
```

Normalization is a crucial step in data preprocessing to enhance the effectiveness and interpretability of machine learning models. Hence MinMaxScaler from sklearn is imported to scale the dataset from 0 to 1. Using the sklearn fit_transform() method, the training dataset is scaled.

In [96]:
```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0,1))
# scaling dataset
scaled_train = scaler.fit_transform(dataset_train)
```

The same data preprocessing is done for test data

In [97]:
```
# Selecting NATURAL_GAS values
dataset_test = test_df.NATURAL_GAS.values
# Reshaping 1D to 2D array
dataset_test = np.reshape(dataset_test, (-1,1))
# Normalizing values between 0 and 1
scaled_test = scaler.fit_transform(dataset_test)
print(*scaled_test[:5])
```
```
[0.18880239] [0.19516955] [0.18772871] [0.19199605] [0.18814296]
```

The time-series data must be divided into X_train and y_train from the training set and X_test and y_test from the testing set in this phase. It is done to turn time series data into a supervised learning problem that can be utilized to train the model. The loop generates input/output sequences of length 50 while iterating through the time series data. Using this method, we can forecast future values while taking into consideration the data's temporal dependence on prior observations.

In [98]:
```
X_train = []
y_train = []
for i in range(50, len(scaled_train)):
    X_train.append(scaled_train[i-50:i, 0])
    y_train.append(scaled_train[i, 0])
    if i <= 51:
        print(X_train)
        print(y_train)
        print()
```
```
[array([0.03908581, 0.03850353, 0.04054152, 0.03886746, 0.04199723,
       0.0451998 , 0.04403523, 0.04461751, 0.0497125 , 0.0497125 ,
       0.05415241, 0.05662712, 0.06696266, 0.06157653, 0.06470631,
       0.07111143, 0.06434238, 0.07067472, 0.06499745, 0.07445957,
       0.07715263, 0.08151976, 0.07424121, 0.08028241, 0.06718102,
       0.06230439, 0.06557974, 0.06936458, 0.0677633 , 0.06565252,
       0.07125701, 0.06732659, 0.0748235 , 0.07234879, 0.07234879,
       0.0637601 , 0.06485188, 0.06623481, 0.07016522, 0.07620642,
       0.08166533, 0.08559575, 0.08326661, 0.0863236 , 0.08814324,
       0.08443118, 0.07795327, 0.08348497, 0.08281154, 0.0888711 ])]
[0.0851590363199658]

[array([0.03908581, 0.03850353, 0.04054152, 0.03886746, 0.04199723,
       0.0451998 , 0.04403523, 0.04461751, 0.0497125 , 0.0497125 ,
       0.05415241, 0.05662712, 0.06696266, 0.06157653, 0.06470631,
       0.07111143, 0.06434238, 0.07067472, 0.06499745, 0.07445957,
       0.07715263, 0.08151976, 0.07424121, 0.08028241, 0.06718102,
       0.06230439, 0.06557974, 0.06936458, 0.0677633 , 0.06565252,
       0.07125701, 0.06732659, 0.0748235 , 0.07234879, 0.07234879,
       0.0637601 , 0.06485188, 0.06623481, 0.07016522, 0.07620642,
       0.08166533, 0.08559575, 0.08326661, 0.0863236 , 0.08814324,
       0.08443118, 0.07795327, 0.08348497, 0.08281154, 0.0888711 ]), array([0.03850353, 0.04054152, 0.03886746, 0.04199723, 0.0451998 ,
       0.04403523, 0.04461751, 0.0497125 , 0.0497125 , 0.05415241,
       0.05662712, 0.06696266, 0.06157653, 0.06470631, 0.07111143,
       0.06434238, 0.07067472, 0.06499745, 0.07445957, 0.07715263,
       0.08151976, 0.07424121, 0.08028241, 0.06718102, 0.06230439,
       0.06557974, 0.06936458, 0.0677633 , 0.06565252, 0.07125701,
       0.06732659, 0.0748235 , 0.07234879, 0.07234879, 0.0637601 ,
       0.06485188, 0.06623481, 0.07016522, 0.07620642, 0.08166533,
       0.08559575, 0.08326661, 0.0863236 , 0.08814324, 0.08443118,
       0.07795327, 0.08348497, 0.08281154, 0.0888711 , 0.08515904])]
[0.0851590363199658, 0.08930780989827105]
```

In [99]:
```
X_test = []
y_test = []
for i in range(50, len(scaled_test)):
    X_test.append(scaled_test[i-50:i, 0])
    y_test.append(scaled_test[i, 0])
```

In this step, the data is converted into a format that is suitable for input to an RNN. np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1)) transforms the X_train array, which was originally a 2-dimensional array of shape (samples, features), into a 3-dimensional array of shape (samples, time steps, features), where time steps denotes the number of time steps in the input sequence and features denotes the number of features in the input data. Size 1 is an additional dimension that serves as an indication that each time step only has a single feature.

The y_train array is transformed from a 1-dimensional array of shape (samples) into a 2-dimensional array of shape (samples, 1) by np.reshape(y_train, (y_train.shape[0], 1)), where each row represents the output value at a certain time step.

In [100]:
```
# The data is converted to Numpy array
X_train, y_train = np.array(X_train), np.array(y_train)

#Reshaping
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1],1))
y_train = np.reshape(y_train, (y_train.shape[0],1))
print("X_train :",X_train.shape,"y_train :",y_train.shape)
```
```
X_train : (4779, 50, 1) y_train : (4779, 1)
```

In [101]:
```
# The data is converted to numpy array
X_test, y_test = np.array(X_test), np.array(y_test)

#Reshaping
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1],1))
y_test = np.reshape(y_test, (y_test.shape[0],1))
print("X_test :",X_test.shape,"y_test :",y_test.shape)
```
```
X_test : (1157, 50, 1) y_test : (1157, 1)
```

In [102]:
```
#pip install tensorflow
```

In [103]:
```
#pip install keras
```

The libraries needed for the model is imported.

In [104]:
```
import warnings
warnings.filterwarnings("ignore")
# importing libraries
import tensorflow as tf
from tensorflow.keras import Model
from keras.models import Sequential
from keras.layers import LSTM
from keras.layers import Dense
from keras.layers import Dropout
from keras.optimizers import SGD
from sklearn import metrics
from sklearn.metrics import mean_squared_error
```
```
WARNING:tensorflow:From C:\Users\Freddy\anaconda3\Lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.
```

This code creates a LSTM Model with three layers and a dense output layer. It employs the optimizer as Adam, mean squared error as the loss function, and accuracy as the evaluation metric while compiling. With a batch size of 1, it fits the model to the training data for 10 epochs. The number of parameters in each layer and the overall number of parameters in the model are listed in a summary of the model architecture.

In [105]:
```
# initializing the model
regressorLSTM = Sequential()

#Adding LSTM layers
regressorLSTM.add(LSTM(50,
                       return_sequences = True,
                       input_shape = (X_train.shape[1],1)))
regressorLSTM.add(LSTM(50,
                       return_sequences = False))
regressorLSTM.add(Dense(25))

#Adding the output layer
regressorLSTM.add(Dense(1))

#Compiling the model
regressorLSTM.compile(optimizer = 'adam',
                      loss = 'mean_squared_error',
                      metrics = ["accuracy"])

#Fitting the model
regressorLSTM.fit(X_train,
                  y_train,
                  batch_size = 1,
                  epochs = 12)
regressorLSTM.summary()
```
```
WARNING:tensorflow:From C:\Users\Freddy\anaconda3\Lib\site-packages\keras\src\backend.py:873: The name tf.get_default_graph is deprecated. Please use tf.compat.v1.get_default_graph instead.

WARNING:tensorflow:From C:\Users\Freddy\anaconda3\Lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.

Epoch 1/12
WARNING:tensorflow:From C:\Users\Freddy\anaconda3\Lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\Freddy\anaconda3\Lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

4779/4779 [==============================] - 159s 33ms/step - loss: 0.0011 - accuracy: 4.1850e-04
Epoch 2/12
4779/4779 [==============================] - 225s 47ms/step - loss: 4.8736e-04 - accuracy: 4.1850e-04
Epoch 3/12
4779/4779 [==============================] - 127s 27ms/step - loss: 4.1192e-04 - accuracy: 4.1850e-04
Epoch 4/12
4779/4779 [==============================] - 126s 26ms/step - loss: 3.7499e-04 - accuracy: 4.1850e-04
Epoch 5/12
4779/4779 [==============================] - 125s 26ms/step - loss: 3.4430e-04 - accuracy: 4.1850e-04
Epoch 6/12
4779/4779 [==============================] - 126s 26ms/step - loss: 3.2627e-04 - accuracy: 4.1850e-04
Epoch 7/12
4779/4779 [==============================] - 126s 26ms/step - loss: 3.1386e-04 - accuracy: 4.1850e-04
Epoch 8/12
4779/4779 [==============================] - 127s 26ms/step - loss: 3.1031e-04 - accuracy: 4.1850e-04
Epoch 9/12
4779/4779 [==============================] - 127s 27ms/step - loss: 3.1180e-04 - accuracy: 4.1850e-04
Epoch 10/12
4779/4779 [==============================] - 125s 26ms/step - loss: 2.9701e-04 - accuracy: 4.1850e-04
Epoch 11/12
4779/4779 [==============================] - 128s 27ms/step - loss: 2.7712e-04 - accuracy: 4.1850e-04
Epoch 12/12
4779/4779 [==============================] - 129s 27ms/step - loss: 2.8334e-04 - accuracy: 4.1850e-04
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 50, 50)            10400

 lstm_1 (LSTM)               (None, 50)                20200

 dense (Dense)               (None, 25)                1275

 dense_1 (Dense)             (None, 1)                 26

=================================================================
Total params: 31901 (124.61 KB)
Trainable params: 31901 (124.61 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

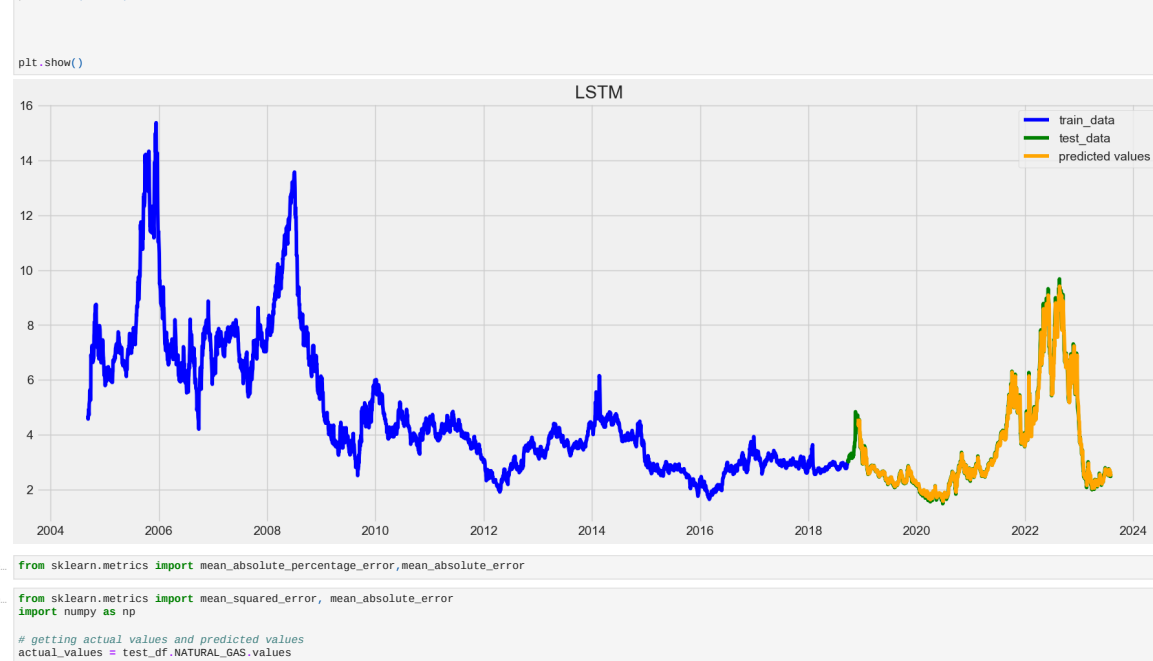
The X_test data is then used to make predictions from the lstm model.

In [106]:
```
#y_lstm = regressor.predict(X_test)
y_LSTM = regressorLSTM.predict(X_test)
```
```
37/37 [==============================] - 2s 21ms/step
```

The predicted values are transformed back from the normalized state to their original scale using the inverse_transform() function.

In [107]:
```
# scaling back from 0-1 to original
prediction = scaler.inverse_transform(y_LSTM)
```

In [108]:
```
plt.figure(figsize=(20,8))
fig.suptitle('Model Predictions')

#Plot for LSTM predictions
plt.plot(train_df.index[1200:], train_df.NATURAL_GAS[1200:], label = "train_data", color = "b")
plt.plot(test_df.index, test_df.NATURAL_GAS, label = "test_data", color = "g")
plt.plot(test_df.index[50:], prediction, label = "predicted values", color = "orange")
plt.legend()
plt.title("LSTM")



plt.show()
```



In [109]:
```
from sklearn.metrics import mean_absolute_percentage_error,mean_absolute_error
```

In [110]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

# getting actual values and predicted values
actual_values = test_df.NATURAL_GAS.values
predicted_values = prediction

# the lengths of arrays are adjusted
min_length = min(len(actual_values), len(predicted_values))
actual_values = actual_values[:min_length]
predicted_values = predicted_values[:min_length]

# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(actual_values, predicted_values)

# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(actual_values, predicted_values)

# Calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# Calculate Mean Absolute Percentage Error (MAPE)
mape = np.mean(np.abs((actual_values - predicted_values) / actual_values)) * 100

print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Percentage Error (MAPE): {mape}%")
```
```
Mean Absolute Error (MAE): 0.8533991501428181
Mean Squared Error (MSE): 1.6400345355316094
Root Mean Squared Error (RMSE): 1.280638331275306
Mean Absolute Percentage Error (MAPE): 56.133003685751494%
```

In [ ]: